Guía EC3 F2 A10

Consumo de API REST y CRUD remoto del Administrador de tareas

Diseño Frontend con Frameworks

Dato	Descripción
Proyecto	Administrador de tareas
Tecnologías	React 19, TypeScript, Vite, PNPM, Fetch API, React Router 7.18.4 y CSS
Producto	SPA conectada con la API REST de Express y MongoDB Atlas
Prerrequisito	Actividad 9 funcional y API Backend de tareas disponible
Modalidad	Demostración, práctica guiada, comprobación y desafío individual

Propósito. Sustituir la persistencia local del Administrador de tareas por solicitudes HTTP reales, conservar la navegación de la Actividad 9 y sincronizar la interfaz React con la API REST desarrollada en Programación Backend con Frameworks.

1 Propósito de la actividad

En la Actividad 9 la aplicación se convirtió en una SPA con páginas, layout, rutas dinámicas y navegación programática, pero las tareas continuaron almacenadas en localStorage. En esta actividad la fuente de verdad será la API REST. La interfaz consultará, creará, actualizará y eliminará documentos persistidos en MongoDB Atlas mediante Fetch API.

El cambio no consiste en insertar llamadas fetch dentro de cada componente. Primero se definirá el contrato HTTP, después se construirá una capa de servicios y finalmente se conectarán páginas y componentes. Esta separación permitirá que la interfaz continúe siendo comprensible y facilitará la incorporación de autenticación en la Actividad 11.

2 Resultados esperados

- Configurar la dirección de la API mediante una variable de entorno de Vite.
- Utilizar un proxy de desarrollo para comunicar los puertos 5173 y 3000 sin modificar el Backend durante la práctica local.
- Crear un cliente HTTP que valide response.ok, interprete JSON, maneje respuestas 204 y normalice errores.
- Implementar GET, POST, PATCH y DELETE mediante un servicio de tareas.
- Consultar tareas con búsqueda, filtro, ordenamiento y paginación del servidor.
- Representar estados de carga, éxito, vacío y error en la interfaz.
- Cancelar solicitudes obsoletas con AbortController.
- Consultar directamente una tarea desde la ruta dinámica /tasks/:taskId.
- Conservar validación, confirmación de eliminación, navegación y accesibilidad.
- Compilar, documentar y registrar la integración en Git.

2.1 Alcance y límites

Esta actividad incorpora:

- Fetch API y async/await.
- Variables VITE_API_URL y configuración del proxy local.
- Contrato paginado data y meta.
- Servicio HTTP reutilizable y servicio específico de tareas.
- Hooks useEffect y useCallback para sincronización remota.
- AbortController para evitar resultados fuera de orden.

- Operaciones CRUD remotas y actualización posterior de la lista.
- Mensajes comprensibles y opción de reintentar.

No se implementarán todavía:

- Registro, inicio de sesión o cierre de sesión.
- Tokens JWT o encabezado Authorization.
- Rutas protegidas o roles.
- Axios, TanStack Query, Redux u otra biblioteca de estado.
- Despliegue de producción y configuración definitiva de CORS.
- Loaders o actions de React Router.

3 Continuidad del proyecto

Actividad	Resultado
Actividad 8	CRUD local, validación, búsqueda, filtros y persistencia local
Actividad 9	Navegación, páginas, layout, rutas dinámicas y 404
Actividad 10	Consumo de API REST y CRUD remoto
Actividad 11	Autenticación, sesión y rutas protegidas
Actividad 12	Integración final, pruebas y despliegue

4 Historias de usuario

ID	Historia	Criterio principal
HU20	Consultar tareas remotas	La lista proviene de GET /api/tasks
HU21	Registrar una tarea	POST crea y la interfaz actualiza los resultados
HU22	Actualizar una tarea	PATCH modifica título o estado persistido
HU23	Eliminar una tarea	DELETE elimina después de confirmar
HU24	Consultar el detalle	La ruta recupera la tarea por su identificador
HU25	Reconocer el estado de la solicitud	La interfaz comunica carga, vacío y error
HU26	Reintentar una consulta	El usuario puede repetir la solicitud fallida

5 Contrato que debe respetarse

5.1 Modelo de tarea

Contrato TypeScript de referencia

```
export type TaskStatus = 'pending' | 'completed';

export interface Task {
  id: string;
  title: string;
  status: TaskStatus;
  createdAt: string;
  updatedAt: string;
}
```

5.2 Rutas utilizadas

Método	Ruta	Uso	Código esperado
GET	/api/tasks	Consultar lista y metadatos	200
GET	/api/tasks/:id	Consultar una tarea	200 o 404
POST	/api/tasks	Crear con title	201
PATCH	/api/tasks/:id	Actualizar title o status	200
DELETE	/api/tasks/:id	Eliminar una tarea	204

5.3 Consulta paginada

Ejemplo de respuesta de GET /api/tasks

```
{
  "data": [
    {
      "id": "IDENTIFICADOR",
      "title": "Preparar entrega",
      "status": "pending",
      "createdAt": "2026-10-14T14:00:00.000Z",
      "updatedAt": "2026-10-14T14:00:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 5,
    "total": 1,
    "totalPages": 1,
    "hasNextPage": false,
    "hasPreviousPage": false
  }
}
```

5.4 Errores de la API

Ejemplo de error uniforme

```
{
  "error": {
    "code": "TASK_NOT_FOUND",
    "message": "La tarea no fue encontrada.",
    "requestId": "IDENTIFICADOR_DE_SOLICITUD"
  }
}
```

La interfaz debe mostrar un mensaje útil. requestId se conserva porque permite relacionar el error presentado al usuario con los registros del servidor sin exponer detalles internos.

6 Conceptos fundamentales

6.1 Solicitud y respuesta HTTP

Una solicitud combina método, dirección, encabezados y, cuando corresponde, un cuerpo. La respuesta contiene un código de estado, encabezados y posiblemente datos. GET consulta, POST crea, PATCH actualiza parcialmente y DELETE elimina.

6.2 Fetch no rechaza automáticamente un 404

fetch devuelve una promesa rechazada ante problemas como una conexión interrumpida, pero una respuesta HTTP 404 o 500 sigue siendo una respuesta válida. Por eso el cliente debe comprobar response.ok y convertir los estados no satisfactorios en errores controlados.

6.3 Variables de entorno en Vite

Vite expone al código del navegador únicamente las variables que comienzan con VITE_. Estas variables terminan dentro del paquete Frontend y nunca deben contener contraseñas, cadenas de MongoDB ni secretos. En esta actividad VITE_API_URL solo contiene la ruta base pública de la API.

6.4 Proxy de desarrollo

El navegador abre el Frontend en el puerto 5173 y la API en el 3000. El proxy de Vite recibe las solicitudes dirigidas a /api y las reenvía al Backend durante pnpm dev. Esto evita resolver CORS dentro de esta actividad. El proxy no forma parte del build de producción; esa configuración se realizará en la Actividad 12.

6.5 Efectos y cancelación

useEffect permite sincronizar un componente con un sistema externo. Cuando cambian búsqueda, filtro, orden o página, se inicia otra consulta. La función de limpieza aborta la solicitud anterior para impedir que una respuesta antigua sustituya resultados más recientes.

6.6 Estados remotos

Estado	Significado	Representación
idle	Todavía no inicia la consulta	Sin mensaje
loading	La solicitud está en curso	Buscando tareas
success	La respuesta contiene resultados	Lista y metadatos
empty	La respuesta contiene data vacío	Mensaje sin resultados
error	La operación no pudo completarse	Mensaje y botón Reintentar

7 Herramientas y prerrequisitos

- Visual Studio Code.
- Node.js y PNPM configurados.
- Proyecto Frontend al finalizar la Actividad 9.
- Proyecto task-api-express al finalizar Backend EC2 F3 A8 o una versión compatible.
- MongoDB Atlas disponible.
- Navegador con herramientas de desarrollo.
- Git y repositorios actualizados.

Parte A Preparación de la integración

Paso 1 Recuperar la versión funcional

Propósito. Evitar integrar la API sobre cambios sin comprobar.

Ubicación. Raíz del proyecto Frontend.

Acción. Abre el repositorio de la Actividad 9, revisa el estado y compila.

Comandos

```
git status
git log --oneline -5
pnpm install
pnpm build
```

Continúa únicamente cuando la compilación termine sin errores y no existan cambios pendientes desconocidos.

Paso 2 Crear la rama de integración

Propósito. Separar el consumo de la API de la versión estable de navegación.

Ubicación. Terminal de Visual Studio Code en la raíz del Frontend.

Acción. Crea una rama y comprueba su nombre.

Comandos

```
git switch -c feature/api-rest-crud-remoto
git branch --show-current
```

Paso 3 Comprobar la continuidad de NotFoundPage

Propósito. Confirmar que el JSX heredado contiene un solo cierre de section.

Archivo. src/pages/NotFoundPage.tsx

Acción. Ejecuta pnpm build. Si existe un cierre duplicado </section>, elimínalo y conserva la estructura equivalente a la siguiente.

Fragmento correcto de NotFoundPage.tsx

```
return (
  <section
    className="panel not-found"
    aria-labelledby="not-found-title"
  >
    {/* Contenido de la página */}
  </section>
);
```

Paso 4 Iniciar y probar la API

Propósito. Aislar problemas del servidor antes de modificar React.

Ubicación. Proyecto task-api-express y Postman.

Acción. Inicia el Backend, comprueba Atlas y ejecuta las rutas principales.

Comando del Backend

```
pnpm install
pnpm dev
```

Comprobación	Dirección	Esperado
Servidor	http://localhost:3000/health	200
Base de datos	http://localhost:3000/health/database	200
Listado	http://localhost:3000/api/tasks?page=1&limit=5	200 con data y meta

Si estas rutas no funcionan en Postman, corrige el Backend antes de continuar. React no debe utilizarse para diagnosticar una API que todavía no responde correctamente.

Paso 5 Declarar la ruta pública de la API

Propósito. Evitar direcciones dispersas dentro del código.

Archivo. .env.example y .env.local

Acción. Crea ambos archivos con el mismo valor local. Conserva .env.local fuera de Git.

Contenido de .env.example

```
VITE_API_URL=/api
```

Contenido de .env.local

```
VITE_API_URL=/api
```

VITE_API_URL no es un secreto. Las credenciales de Atlas pertenecen exclusivamente al archivo .env del Backend y nunca deben copiarse al Frontend.

Paso 6 Configurar el proxy de desarrollo

Propósito. Reenviar /api al servidor Express mientras se utiliza pnpm dev.

Archivo. vite.config.ts

Acción. Reemplaza el contenido por la configuración equivalente.

Código de vite.config.ts

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      '/api': {
        target: 'http://localhost:3000',
        changeOrigin: true,
      },
    },
  },
});
```

Después de modificar vite.config.ts debes reiniciar pnpm dev. El proxy solo funciona a través del servidor de desarrollo de Vite.

Paso 7 Tipar la variable de entorno

Propósito. Obtener autocompletado y validación en TypeScript.

Archivo. src/vite-env.d.ts

Acción. Conserva la referencia de Vite y agrega las declaraciones.

Código de src/vite-env.d.ts

```
/// <reference types="vite/client" />

interface ImportMetaEnv {
  readonly VITE_API_URL: string;
}

interface ImportMeta {
  readonly env: ImportMetaEnv;
}
```

Parte B Contratos y servicios HTTP

Paso 8 Completar el modelo de tareas

Propósito. Compartir tipos de entrada con el servicio remoto.

Archivo. src/models/task.interface.ts

Acción. Reemplaza el contenido y conserva exactamente los nombres del Backend.

Código de task.interface.ts

```
export type TaskStatus = 'pending' | 'completed';

export interface Task {
  id: string;
  title: string;
  status: TaskStatus;
  createdAt: string;
  updatedAt: string;
}

export interface CreateTaskInput {
  title: string;
}

export interface UpdateTaskInput {
  title?: string;
  status?: TaskStatus;
}
```

Paso 9 Crear los tipos del contrato HTTP

Propósito. Representar respuestas, metadatos, errores y parámetros de consulta.

Archivo. src/models/api.types.ts

Acción. Crea el archivo completo.

Código de api.types.ts fragmento 1 de 2

```
import type {
  Task,
  TaskStatus,
} from './task.interface';

export interface ApiDataResponse<T> {
  data: T;
}

export interface ApiErrorBody {
  error: {
    code: string;
    message: string;
    details?: unknown;
    requestId?: string;
  };
}

export interface PageMeta {
  page: number;
  limit: number;
  total: number;
  totalPages: number;
  hasNextPage: boolean;
```

Código de api.types.ts fragmento 2 de 2

```
  hasPreviousPage: boolean;
}

export interface TaskListResponse {
  data: Task[];
  meta: PageMeta;
}

export type TaskSortField =
  | 'title'
  | 'status'
  | 'createdAt'
  | 'updatedAt';

export type SortOrder = 'asc' | 'desc';

export interface TaskListQuery {
  page: number;
  limit: number;
  search?: string;
  status?: TaskStatus;
  sortBy: TaskSortField;
  order: SortOrder;
}
```

Paso 10 Leer y validar la configuración

Propósito. Centralizar la dirección base antes de ejecutar solicitudes.

Archivo. src/config/env.ts

Acción. Crea el directorio config y el archivo completo.

Código de env.ts

```
const rawApiUrl =
  import.meta.env.VITE_API_URL?.trim();

if (!rawApiUrl) {
  throw new Error(
    'Falta la variable VITE_API_URL.',
  );
}

export const API_URL = rawApiUrl.replace(
  /\/+$/,
  '',
);
```

Paso 11 Crear el cliente HTTP

Propósito. Reutilizar encabezados, JSON, response.ok, errores y respuestas sin cuerpo.

Archivo. src/services/http.service.ts

Acción. Crea el archivo completo.

Código de http.service.ts fragmento 1 de 4

```
import { API_URL } from '../config/env';
import type { ApiErrorBody } from '../models/api.types';

interface HttpErrorOptions {
  status: number;
  code: string;
  message: string;
  requestId?: string;
  details?: unknown;
}

export class HttpError extends Error {
  readonly status: number;
  readonly code: string;
  readonly requestId?: string;
  readonly details?: unknown;

  constructor(options: HttpErrorOptions) {
    super(options.message);
    this.name = 'HttpError';
    this.status = options.status;
    this.code = options.code;
    this.requestId = options.requestId;
    this.details = options.details;
  }
}

function isRecord(
  value: unknown,
): value is Record<string, unknown> {
  return typeof value === 'object' &&
    value !== null;
}

function isApiErrorBody(
  value: unknown,
): value is ApiErrorBody {
  if (!isRecord(value)) {
    return false;
  }
```

Código de http.service.ts fragmento 2 de 4

```
  const error = value.error;

  return isRecord(error) &&
    typeof error.code === 'string' &&
    typeof error.message === 'string';
}

async function readPayload(
  response: Response,
): Promise<unknown> {
  const contentType =
    response.headers.get('content-type');

  if (!contentType?.includes('application/json')) {
    return null;
  }

  try {
    return await response.json();
  } catch {
    throw new HttpError({
      status: response.status,
      code: 'INVALID_RESPONSE',
      message:
        'La API devolvió una respuesta JSON inválida.',
    });
  }
}

export async function httpRequest<T>(
  path: string,
  options: RequestInit = {},
): Promise<T> {
  const headers = new Headers(options.headers);
  headers.set('Accept', 'application/json');

  if (options.body && !headers.has('Content-Type')) {
    headers.set('Content-Type', 'application/json');
  }
```

Código de http.service.ts fragmento 3 de 4

```
try {
  const response = await fetch(
    `${API_URL}${path}`,
    {
      ...options,
      headers,
    },
  );

  if (response.status === 204) {
    return undefined as T;
  }

  const payload = await readPayload(response);

  if (!response.ok) {
    if (isApiErrorBody(payload)) {
      throw new HttpError({
        status: response.status,
        code: payload.error.code,
        message: payload.error.message,
        requestId: payload.error.requestId,
        details: payload.error.details,
      });
    }

    throw new HttpError({
      status: response.status,
      code: 'HTTP_ERROR',
      message:
        'La API respondió con un error.',
    });
  }

  return payload as T;
} catch (error) {
  if (
    error instanceof HttpError ||
    isAbortError(error)
```

Código de http.service.ts fragmento 4 de 4

```
    ) {
      throw error;
    }

    throw new HttpError({
      status: 0,
      code: 'NETWORK_ERROR',
      message:
        'No fue posible conectar con la API.',
    });
  }
}

export function isAbortError(
  error: unknown,
): boolean {
  return error instanceof DOMException &&
    error.name === 'AbortError';
}

export function isHttpError(
  error: unknown,
): error is HttpError {
  return error instanceof HttpError;
}

export function getHttpErrorMessage(
  error: unknown,
): string {
  if (!isHttpError(error)) {
    return 'Ocurrió un error inesperado.';
  }

  const reference = error.requestId
    ? ` Referencia: ${error.requestId}.`
    : '';

  return `${error.message}${reference}`;
}
```

El cliente no conoce tareas ni componentes. Su responsabilidad es convertir una interacción HTTP en un valor tipado o en HttpError.

Paso 12 Crear el servicio remoto de tareas

Propósito. Concentrar rutas, métodos y parámetros de consulta.

Archivo. src/services/task.api.service.ts

Acción. Crea el archivo completo.

Código de task.api.service.ts fragmento 1 de 2

```
import type {
  ApiDataResponse,
  TaskListQuery,
  TaskListResponse,
} from '../models/api.types';
import type {
  CreateTaskInput,
  Task,
  UpdateTaskInput,
} from '../models/task.interface';
import { httpRequest } from './http.service';

function createQueryString(
  query: TaskListQuery,
): string {
  const params = new URLSearchParams({
    page: String(query.page),
    limit: String(query.limit),
    sortBy: query.sortBy,
    order: query.order,
  });

  if (query.search) {
    params.set('search', query.search);
  }

  if (query.status) {
    params.set('status', query.status);
  }

  return params.toString();
}

export function listTasks(
  query: TaskListQuery,
  signal?: AbortSignal,
): Promise<TaskListResponse> {
  return httpRequest<TaskListResponse>(
    `/tasks?${createQueryString(query)}`,
    { signal },
  );
}
```

Código de task.api.service.ts fragmento 2 de 2

```
export function getTaskById(
  id: string,
  signal?: AbortSignal,
): Promise<ApiDataResponse<Task>> {
  return httpRequest<ApiDataResponse<Task>>(
    `/tasks/${encodeURIComponent(id)}`,
    { signal },
  );
}

export function createTask(
  input: CreateTaskInput,
): Promise<ApiDataResponse<Task>> {
  return httpRequest<ApiDataResponse<Task>>(
    '/tasks',
    {
      method: 'POST',
      body: JSON.stringify(input),
    },
  );
}

export function updateTask(
  id: string,
  input: UpdateTaskInput,
): Promise<ApiDataResponse<Task>> {
  return httpRequest<ApiDataResponse<Task>>(
    `/tasks/${encodeURIComponent(id)}`,
    {
      method: 'PATCH',
      body: JSON.stringify(input),
    },
  );
}

export function deleteTask(
  id: string,
): Promise<void> {
  return httpRequest<void>(
    `/tasks/${encodeURIComponent(id)}`,
    { method: 'DELETE' },
  );
}
```

URLSearchParams codifica correctamente los valores de la consulta. Los componentes no deben construir direcciones ni repetir JSON.stringify.

Parte C Estado remoto y componentes

Paso 13 Definir los estados de solicitud

Propósito. Utilizar valores controlados para representar el ciclo asíncrono.

Archivo. src/models/request-status.type.ts

Acción. Crea el archivo completo.

Código de request-status.type.ts

```
export type RequestStatus =
  | 'idle'
  | 'loading'
  | 'success'
  | 'empty'
  | 'error';
```

Paso 14 Crear el hook de consulta

Propósito. Solicitar la lista, conservar metadatos, cancelar solicitudes y permitir reintentos.

Archivo. src/hooks/useTaskList.ts

Acción. Crea el directorio hooks y el archivo completo.

Código de useTaskList.ts fragmento 1 de 3

```
import {
  useCallback,
  useEffect,
  useState,
} from 'react';
import type {
  PageMeta,
  TaskListQuery,
} from '../models/api.types';
import type { RequestStatus } from '../models/request-status.type';
import type { Task } from '../models/task.interface';
import {
  getHttpErrorMessage,
  isAbortError,
} from '../services/http.service';
import { listTasks } from '../services/task.api.service';

interface TaskListState {
  tasks: Task[];
  meta: PageMeta;
  status: RequestStatus;
  errorMessage: string;
}

const EMPTY_META: PageMeta = {
  page: 1,
  limit: 5,
  total: 0,
  totalPages: 0,
  hasNextPage: false,
  hasPreviousPage: false,
};

export function useTaskList(
  query: TaskListQuery,
) {
  const {
    page,
    limit,
    search,
    status: statusFilter,
    sortBy,
    order,
  } = query;
```

Código de useTaskList.ts fragmento 2 de 3

```
const [state, setState] =
  useState<TaskListState>({
    tasks: [],
    meta: {
      ...EMPTY_META,
      page,
      limit,
    },
    status: 'idle',
    errorMessage: '',
  });

const [reloadKey, setReloadKey] =
  useState(0);

const reload = useCallback(() => {
  setReloadKey((current) => current + 1);
}, []);

useEffect(() => {
  const controller = new AbortController();

  async function load(): Promise<void> {
    setState((current) => ({
      ...current,
      status: 'loading',
      errorMessage: '',
    }));

    try {
      const response = await listTasks({
        page,
        limit,
        search,
        status: statusFilter,
        sortBy,
        order,
      }, controller.signal);

      setState({
        tasks: response.data,
        meta: response.meta,
        status:
```

Código de useTaskList.ts fragmento 3 de 3

```
            response.data.length === 0
              ? 'empty'
              : 'success',
          errorMessage: '',
        });
      } catch (error) {
        if (isAbortError(error)) {
          return;
        }

        setState({
          tasks: [],
          meta: {
            ...EMPTY_META,
            page,
            limit,
          },
          status: 'error',
          errorMessage:
            getHttpErrorMessage(error),
        });
      }
    }

    void load();

    return () => {
      controller.abort();
    };
  }, [
    page,
    limit,
    search,
    statusFilter,
    sortBy,
    order,
    reloadKey,
  ]);

  return {
    ...state,
    reload,
  };
}
```

React StrictMode puede iniciar, limpiar y repetir un efecto durante el desarrollo. La cancelación evita que la primera solicitud actualice el estado después de quedar obsoleta.

Paso 15 Crear el componente de estado remoto

Propósito. Comunicar carga, ausencia de resultados y fallas recuperables.

Archivo. src/components/AsyncStatus.tsx

Acción. Crea el archivo completo.

Código de AsyncStatus.tsx fragmento 1 de 2

```
import type { RequestStatus } from '../models/request-status.type';

interface AsyncStatusProps {
  status: RequestStatus;
  errorMessage: string;
  onRetry: () => void;
}

export function AsyncStatus({
  status,
  errorMessage,
  onRetry,
}: AsyncStatusProps) {
  if (status === 'idle' || status === 'success') {
    return null;
  }

  if (status === 'loading') {
    return (
      <p className="request-status" role="status">
        Consultando tareas...
      </p>
    );
  }
```

Código de AsyncStatus.tsx fragmento 2 de 2

```
  if (status === 'empty') {
    return (
      <p className="request-status" role="status">
        No existen tareas que coincidan con la consulta.
      </p>
    );
  }

  return (
    <section
      className="request-status request-status--error"
      aria-labelledby="request-error-title"
    >
      <h3 id="request-error-title">
        No fue posible consultar las tareas
      </h3>

      <p>{errorMessage}</p>

      <button type="button" onClick={onRetry}>
        Reintentar
      </button>
    </section>
  );
}
```

Paso 16 Crear la paginación remota

Propósito. Cambiar de página utilizando los metadatos del servidor.

Archivo. src/components/TaskPagination.tsx

Acción. Crea o reemplaza el archivo completo.

Código de TaskPagination.tsx fragmento 1 de 2

```
import type { PageMeta } from '../models/api.types';

interface TaskPaginationProps {
  meta: PageMeta;
  disabled: boolean;
  onPageChange: (page: number) => void;
}

export function TaskPagination({
  meta,
  disabled,
  onPageChange,
}: TaskPaginationProps) {
  if (meta.totalPages <= 1) {
    return null;
  }

  return (
    <nav
      className="pagination"
      aria-label="Paginación de tareas"
    >
      <button
        type="button"
        disabled={
```

Código de TaskPagination.tsx fragmento 2 de 2

```
          disabled || !meta.hasPreviousPage
        }
        onClick={() =>
          onPageChange(meta.page - 1)
        }
      >
        Anterior
      </button>

      <span aria-live="polite">
        Página {meta.page} de {meta.totalPages}
      </span>

      <button
        type="button"
        disabled={disabled || !meta.hasNextPage}
        onClick={() =>
          onPageChange(meta.page + 1)
        }
      >
        Siguiente
      </button>
    </nav>
  );
}
```

Paso 17 Actualizar el resumen

Propósito. Diferenciar el total remoto de los registros visibles en la página.

Archivo. src/components/TaskSummary.tsx

Acción. Reemplaza todo el contenido.

Código de TaskSummary.tsx

```
import type { Task } from '../models/task.interface';

interface TaskSummaryProps {
  tasks: Task[];
  total: number;
}

export function TaskSummary({
  tasks,
  total,
}: TaskSummaryProps) {
  const pendingOnPage = tasks.filter(
    (task) => task.status === 'pending',
  ).length;

  const completedOnPage =
    tasks.length - pendingOnPage;

  return (
    <section
      className="task-summary"
      aria-label="Resumen de tareas"
    >
      <p>
        <strong>{total}</strong> resultado(s) en la consulta
      </p>

      <p>
        Página actual: {pendingOnPage} pendiente(s) y{' '}
        {completedOnPage} completada(s)
      </p>
    </section>
  );
}
```

La API no entrega totales separados por estado. Por eso el resumen indica claramente que los conteos pending y completed corresponden solo a la página actual.

Paso 18 Actualizar el formulario

Propósito. Esperar la respuesta POST y limpiar el campo únicamente cuando la creación sea satisfactoria.

Archivo. src/components/TaskForm.tsx

Acción. Reemplaza todo el contenido.

Código de TaskForm.tsx fragmento 1 de 2

```
import { useState } from 'react';
import type { FormEvent } from 'react';
import {
  TASK_TITLE_MAX_LENGTH,
  validateTaskTitle,
} from '../utils/task-validation';

interface TaskFormProps {
  isSubmitting: boolean;
  onAddTask: (title: string) => Promise<boolean>;
}

export function TaskForm({
  isSubmitting,
  onAddTask,
}: TaskFormProps) {
  const [title, setTitle] = useState('');
  const [error, setError] = useState('');

  function handleTitleChange(value: string): void {
    setTitle(value);

    if (error) {
      setError(validateTaskTitle(value) ?? '');
    }
  }

  async function handleSubmit(
    event: FormEvent<HTMLFormElement>,
  ): Promise<void> {
    event.preventDefault();

    const validationMessage =
      validateTaskTitle(title);

    if (validationMessage) {
      setError(validationMessage);
      return;
    }

    const wasCreated = await onAddTask(
      title.trim(),
    );

    if (wasCreated) {
      setTitle('');
```

Código de TaskForm.tsx fragmento 2 de 2

```
      setError('');
    }
  }

  return (
    <form
      className="task-form"
      onSubmit={handleSubmit}
      noValidate
    >
      <label htmlFor="task-title">
        Nueva tarea
      </label>

      <div className="task-form__row">
        <input
          id="task-title"
          value={title}
          maxLength={TASK_TITLE_MAX_LENGTH}
          disabled={isSubmitting}
          onChange={(event) =>
            handleTitleChange(event.target.value)
          }
          aria-invalid={Boolean(error)}
          aria-describedby={
            error ? 'task-title-error' : undefined
          }
        />

        <button type="submit" disabled={isSubmitting}>
          {isSubmitting ? 'Guardando...' : 'Agregar'}
        </button>
      </div>

      {error && (
        <p
          id="task-title-error"
          className="field-error"
          role="alert"
        >
          {error}
        </p>
      )}
    </form>
  );
}
```

Paso 19 Actualizar los filtros

Propósito. Enviar criterios compatibles con la consulta del Backend y limitar search a 60 caracteres.

Archivo. src/components/TaskFilters.tsx

Acción. Reemplaza todo el contenido.

Código de TaskFilters.tsx fragmento 1 de 3

```
import type {
  TaskFilter,
  TaskSort,
} from '../models/task-view.types';

interface TaskFiltersProps {
  query: string;
  filter: TaskFilter;
  sortBy: TaskSort;
  resultCount: number;
  isLoading: boolean;
  onQueryChange: (value: string) => void;
  onFilterChange: (value: TaskFilter) => void;
  onSortChange: (value: TaskSort) => void;
  onClear: () => void;
}

export function TaskFilters({
  query,
  filter,
  sortBy,
  resultCount,
  isLoading,
  onQueryChange,
  onFilterChange,
  onSortChange,
  onClear,
}: TaskFiltersProps) {
  return (
    <section
      className="task-filters"
      aria-labelledby="filters-title"
    >
      <h3 id="filters-title">Consultar tareas</h3>
```

Código de TaskFilters.tsx fragmento 2 de 3

```
      <div className="task-filters__grid">
        <label>
          Buscar
          <input
            type="search"
            value={query}
            maxLength={60}
            placeholder="Ejemplo: preparar"
            onChange={(event) =>
              onQueryChange(event.target.value)
            }
          />
        </label>

        <label>
          Estado
          <select
            value={filter}
            onChange={(event) =>
              onFilterChange(
                event.target.value as TaskFilter,
              )
            }
          >
            <option value="all">Todas</option>
            <option value="pending">Pendientes</option>
            <option value="completed">Completadas</option>
          </select>
        </label>

        <label>
          Orden
          <select
            value={sortBy}
            onChange={(event) =>
```

Código de TaskFilters.tsx fragmento 3 de 3

```
              onSortChange(
                event.target.value as TaskSort,
              )
            }
          >
            <option value="updated-desc">
              Actualizadas recientemente
            </option>
            <option value="updated-asc">
              Actualizadas anteriormente
            </option>
            <option value="title-asc">
              Título A-Z
            </option>
            <option value="title-desc">
              Título Z-A
            </option>
          </select>
        </label>
      </div>

      <div className="task-filters__footer">
        <p role="status">
          {isLoading
            ? 'Actualizando resultados...'
            : `${resultCount} resultado(s)`}
        </p>

        <button type="button" onClick={onClear}>
          Limpiar criterios
        </button>
      </div>
    </section>
  );
}
```

Paso 20 Actualizar cada tarea

Propósito. Esperar PATCH y DELETE antes de cerrar la edición o habilitar nuevamente los controles.

Archivo. src/components/TaskItem.tsx

Acción. Reemplaza todo el contenido.

Código de TaskItem.tsx fragmento 1 de 5

```
import { useState } from 'react';
import type { FormEvent } from 'react';
import { Link } from 'react-router';
import type { Task } from '../models/task.interface';
import { formatTaskDate } from '../utils/date';
import {
  TASK_TITLE_MAX_LENGTH,
  validateTaskTitle,
} from '../utils/task-validation';

interface TaskItemProps {
  task: Task;
  onToggleStatus: (task: Task) => Promise<boolean>;
  onEditTask: (
    task: Task,
    title: string,
  ) => Promise<boolean>;
  onDeleteTask: (task: Task) => Promise<boolean>;
}

export function TaskItem({
  task,
  onToggleStatus,
  onEditTask,
  onDeleteTask,
}: TaskItemProps) {
  const [isEditing, setIsEditing] = useState(false);
  const [draftTitle, setDraftTitle] =
    useState(task.title);
  const [error, setError] = useState('');
  const [isBusy, setIsBusy] = useState(false);

  const statusLabel =
    task.status === 'completed'
      ? 'Completada'
      : 'Pendiente';

  function startEditing(): void {
    setDraftTitle(task.title);
    setError('');
```

Código de TaskItem.tsx fragmento 2 de 5

```
    setIsEditing(true);
  }

  function cancelEditing(): void {
    setDraftTitle(task.title);
    setError('');
    setIsEditing(false);
  }

  async function handleEditSubmit(
    event: FormEvent<HTMLFormElement>,
  ): Promise<void> {
    event.preventDefault();

    const validationMessage =
      validateTaskTitle(draftTitle);

    if (validationMessage) {
      setError(validationMessage);
      return;
    }

    setIsBusy(true);
    const wasUpdated = await onEditTask(
      task,
      draftTitle.trim(),
    );
    setIsBusy(false);

    if (wasUpdated) {
      setIsEditing(false);
      setError('');
    }
  }

  async function handleToggle(): Promise<void> {
    setIsBusy(true);
    await onToggleStatus(task);
    setIsBusy(false);
  }
```

Código de TaskItem.tsx fragmento 3 de 5

```
async function handleDelete(): Promise<void> {
  setIsBusy(true);
  await onDeleteTask(task);
  setIsBusy(false);
}

return (
  <article
    className={`task-item task-item--${task.status}`}
    role="listitem"
    aria-busy={isBusy}
  >
    <div className="task-item__content">
      {isEditing ? (
        <form
          className="task-item__edit-form"
          onSubmit={handleEditSubmit}
          noValidate
        >
          <label
            className="sr-only"
            htmlFor={`edit-task-${task.id}`}
          >
            Editar título
          </label>

          <input
            id={`edit-task-${task.id}`}
            value={draftTitle}
            maxLength={TASK_TITLE_MAX_LENGTH}
            disabled={isBusy}
            onChange={(event) =>
              setDraftTitle(event.target.value)
            }
            autoFocus
          />

          {error && (
```

Código de TaskItem.tsx fragmento 4 de 5

```
              <p className="field-error" role="alert">
                {error}
              </p>
            )}

            <div className="task-item__edit-actions">
              <button type="submit" disabled={isBusy}>
                Guardar
              </button>
              <button
                type="button"
                disabled={isBusy}
                onClick={cancelEditing}
              >
                Cancelar
              </button>
            </div>
          </form>
        ) : (
          <>
            <h3>{task.title}</h3>
            <span
              className={`status-badge status-badge--${task.status}`}
            >
              {statusLabel}
            </span>
            <p className="task-item__date">
              Actualizada:{' '}
              <time dateTime={task.updatedAt}>
                {formatTaskDate(task.updatedAt)}
              </time>
            </p>
          </>
        )}
      </div>

      <div
        className="task-item__actions"
        role="group"
```

Código de TaskItem.tsx fragmento 5 de 5

```
        aria-label={`Acciones para ${task.title}`}
      >
        <Link
          className="button-link button-link--secondary"
          to={`/tasks/${task.id}`}
        >
          Ver detalle
        </Link>

        <button
          type="button"
          disabled={isBusy || isEditing}
          onClick={() => void handleToggle()}
        >
          {task.status === 'completed'
            ? 'Reabrir'
            : 'Completar'}
        </button>

        <button
          type="button"
          disabled={isBusy || isEditing}
          onClick={startEditing}
        >
          Editar
        </button>

        <button
          type="button"
          className="button-danger"
          disabled={isBusy || isEditing}
          onClick={() => void handleDelete()}
        >
          Eliminar
        </button>
      </div>
    </article>
  );
}
```

Paso 21 Actualizar la lista

Propósito. Propagar las operaciones asíncronas a cada elemento.

Archivo. src/components/TaskList.tsx

Acción. Reemplaza todo el contenido.

Código de TaskList.tsx

```
import type { Task } from '../models/task.interface';
import { TaskItem } from './TaskItem';

interface TaskListProps {
  tasks: Task[];
  onToggleStatus: (task: Task) => Promise<boolean>;
  onEditTask: (
    task: Task,
    title: string,
  ) => Promise<boolean>;
  onDeleteTask: (task: Task) => Promise<boolean>;
}

export function TaskList({
  tasks,
  onToggleStatus,
  onEditTask,
  onDeleteTask,
}: TaskListProps) {
  if (tasks.length === 0) {
    return null;
  }

  return (
    <div className="task-list" role="list">
      {tasks.map((task) => (
        <TaskItem
          key={task.id}
          task={task}
          onToggleStatus={onToggleStatus}
          onEditTask={onEditTask}
          onDeleteTask={onDeleteTask}
        />
      ))}
    </div>
  );
}
```

Paso 22 Comprobar los tipos de consulta visual

Propósito. Conservar los valores utilizados por filtros y orden.

Archivo. src/models/task-view.types.ts

Acción. Comprueba el contenido y reemplázalo si no coincide.

Código de task-view.types.ts

```
import type { TaskStatus } from './task.interface';

export type TaskFilter = 'all' | TaskStatus;

export type TaskSort =
  | 'updated-desc'
  | 'updated-asc'
  | 'title-asc'
  | 'title-desc';
```

Parte D Páginas y coordinación

Paso 23 Actualizar la página de inicio

Propósito. Explicar que la aplicación ya utiliza persistencia remota.

Archivo. src/pages/HomePage.tsx

Acción. Reemplaza todo el contenido.

Código de HomePage.tsx

```
import { Link } from 'react-router';

export function HomePage() {
  return (
    <section
      className="panel welcome-panel"
      aria-labelledby="home-title"
    >
      <p className="section-heading__eyebrow">
        Inicio
      </p>

      <h2 id="home-title">
        Administrador de tareas conectado
      </h2>

      <p>
        Las tareas se consultan y actualizan mediante
        la API REST de Express y MongoDB Atlas.
      </p>

      <div className="welcome-panel__actions">
        <Link className="button-link" to="/tasks">
          Administrar tareas
        </Link>
      </div>
    </section>
  );
}
```

Paso 24 Actualizar la página de tareas

Propósito. Coordinar consulta remota, filtros, paginación y mutaciones sin devolver responsabilidades a App.

Archivo. src/pages/TasksPage.tsx

Acción. Reemplaza todo el contenido.

Código de TasksPage.tsx fragmento 1 de 7

```
import { useState } from 'react';
import { AsyncStatus } from '../components/AsyncStatus';
import { TaskFilters } from '../components/TaskFilters';
import { TaskForm } from '../components/TaskForm';
import { TaskList } from '../components/TaskList';
import { TaskPagination } from '../components/TaskPagination';
import { TaskSummary } from '../components/TaskSummary';
import { useTaskList } from '../hooks/useTaskList';
import type {
  TaskListQuery,
  TaskSortField,
  SortOrder,
} from '../models/api.types';
import type { Task } from '../models/task.interface';
import type {
  TaskFilter,
  TaskSort,
} from '../models/task-view.types';
import { getHttpErrorMessage } from '../services/http.service';
import {
  createTask as createTaskRequest,
  deleteTask as deleteTaskRequest,
  updateTask as updateTaskRequest,
} from '../services/task.api.service';

const PAGE_SIZE = 5;

function mapSort(sort: TaskSort): {
  sortBy: TaskSortField;
  order: SortOrder;
} {
  switch (sort) {
    case 'updated-asc':
      return { sortBy: 'updatedAt', order: 'asc' };
    case 'title-asc':
      return { sortBy: 'title', order: 'asc' };
    case 'title-desc':
      return { sortBy: 'title', order: 'desc' };
    default:
      return { sortBy: 'updatedAt', order: 'desc' };
  }
}
```

Código de TasksPage.tsx fragmento 2 de 7

```
export function TasksPage() {
  const [query, setQuery] = useState('');
  const [filter, setFilter] =
    useState<TaskFilter>('all');
  const [sort, setSort] =
    useState<TaskSort>('updated-desc');
  const [page, setPage] = useState(1);
  const [isMutating, setIsMutating] =
    useState(false);
  const [operationMessage, setOperationMessage] =
    useState('');
  const [operationFailed, setOperationFailed] =
    useState(false);

  const mappedSort = mapSort(sort);

  const taskListQuery: TaskListQuery = {
    page,
    limit: PAGE_SIZE,
    search: query.trim() || undefined,
    status: filter === 'all' ? undefined : filter,
    sortBy: mappedSort.sortBy,
    order: mappedSort.order,
  };

  const {
    tasks,
    meta,
    status,
    errorMessage,
    reload,
  } = useTaskList(taskListQuery);

  async function runMutation(
    operation: () => Promise<unknown>,
    successMessage: string,
  ): Promise<boolean> {
    setIsMutating(true);
    setOperationMessage('');
    setOperationFailed(false);
```

Código de TasksPage.tsx fragmento 3 de 7

```
    try {
      await operation();
      setOperationMessage(successMessage);
      return true;
    } catch (error) {
      setOperationFailed(true);
      setOperationMessage(
        getHttpErrorMessage(error),
      );
      return false;
    } finally {
      setIsMutating(false);
    }
  }

  function refreshAfterMutation(): void {
    if (page !== 1) {
      setPage(1);
      return;
    }

    reload();
  }

  async function handleAddTask(
    title: string,
  ): Promise<boolean> {
    const succeeded = await runMutation(
      () => createTaskRequest({ title }),
      'La tarea fue creada.',
    );

    if (succeeded) {
      refreshAfterMutation();
    }

    return succeeded;
  }

  async function handleToggleStatus(
    task: Task,
  ): Promise<boolean> {
```

Código de TasksPage.tsx fragmento 4 de 7

```
    const nextStatus =
      task.status === 'pending'
        ? 'completed'
        : 'pending';

    const succeeded = await runMutation(
      () => updateTaskRequest(task.id, {
        status: nextStatus,
      }),
      'El estado fue actualizado.',
    );

    if (succeeded) {
      reload();
    }

    return succeeded;
  }

  async function handleEditTask(
    task: Task,
    title: string,
  ): Promise<boolean> {
    const succeeded = await runMutation(
      () => updateTaskRequest(task.id, { title }),
      'El título fue actualizado.',
    );

    if (succeeded) {
      reload();
    }

    return succeeded;
  }

  async function handleDeleteTask(
    task: Task,
  ): Promise<boolean> {
    const shouldDelete = window.confirm(
      `¿Eliminar la tarea "${task.title}"?`,
    );
```

Código de TasksPage.tsx fragmento 5 de 7

```
  if (!shouldDelete) {
    return false;
  }

  const succeeded = await runMutation(
    () => deleteTaskRequest(task.id),
    'La tarea fue eliminada.',
  );

  if (succeeded) {
    if (tasks.length === 1 && page > 1) {
      setPage(page - 1);
    } else {
      reload();
    }
  }

  return succeeded;
}

function handleQueryChange(value: string): void {
  setQuery(value);
  setPage(1);
}

function handleFilterChange(
  value: TaskFilter,
): void {
  setFilter(value);
  setPage(1);
}

function handleSortChange(value: TaskSort): void {
  setSort(value);
  setPage(1);
}

function handleClear(): void {
  setQuery('');
  setFilter('all');
```

Código de TasksPage.tsx fragmento 6 de 7

```
    setSort('updated-desc');
    setPage(1);
    setOperationMessage('');
  }

  return (
    <div className="page-stack">
      <section
        className="page-introduction"
        aria-labelledby="tasks-page-title"
      >
        <p className="section-heading__eyebrow">
          Gestión remota
        </p>
        <h2 id="tasks-page-title">Mis tareas</h2>
        <p>
          Los cambios se guardan mediante la API REST.
        </p>
      </section>

      <TaskForm
        isSubmitting={isMutating}
        onAddTask={handleAddTask}
      />

      {operationMessage && (
        <p
          className={
            operationFailed
              ? 'operation-message operation-message--error'
              : 'operation-message'
          }
          role={operationFailed ? 'alert' : 'status'}
        >
          {operationMessage}
        </p>
      )}

      <TaskFilters
        query={query}
        filter={filter}
```

Código de TasksPage.tsx fragmento 7 de 7

```
        sortBy={sort}
        resultCount={meta.total}
        isLoading={status === 'loading'}
        onQueryChange={handleQueryChange}
        onFilterChange={handleFilterChange}
        onSortChange={handleSortChange}
        onClear={handleClear}
      />

      <AsyncStatus
        status={status}
        errorMessage={errorMessage}
        onRetry={reload}
      />

      {status !== 'error' && (
        <>
          <TaskSummary
            tasks={tasks}
            total={meta.total}
          />

          <TaskList
            tasks={tasks}
            onToggleStatus={handleToggleStatus}
            onEditTask={handleEditTask}
            onDeleteTask={handleDeleteTask}
          />

          <TaskPagination
            meta={meta}
            disabled={
              status === 'loading' || isMutating
            }
            onPageChange={setPage}
          />
        </>
      )}
    </div>
  );
}
```

Cada cambio de consulta restablece page en 1. Si el usuario escribe rápidamente, useTaskList cancela la solicitud anterior y conserva únicamente la respuesta vigente.

Paso 25 Actualizar la página de detalle

Propósito. Consultar GET /api/tasks/:id al abrir o recargar una ruta dinámica.

Archivo. src/pages/TaskDetailPage.tsx

Acción. Reemplaza todo el contenido.

Código de TaskDetailPage.tsx fragmento 1 de 5

```
import {
  useEffect,
  useState,
} from 'react';
import {
  Link,
  useNavigate,
  useParams,
} from 'react-router';
import type { Task } from '../models/task.interface';
import {
  getHttpErrorMessage,
  isAbortError,
  isHttpError,
} from '../services/http.service';
import {
  deleteTask,
  getTaskById,
} from '../services/task.api.service';
import { formatTaskDate } from '../utils/date';
import { NotFoundPage } from './NotFoundPage';

type DetailStatus =
  | 'loading'
  | 'success'
  | 'not-found'
  | 'error';

export function TaskDetailPage() {
  const { taskId } = useParams<{
    taskId: string;
  }>();
  const navigate = useNavigate();
  const [task, setTask] = useState<Task | null>(null);
  const [status, setStatus] =
    useState<DetailStatus>('loading');
  const [errorMessage, setErrorMessage] =
    useState('');
  const [reloadKey, setReloadKey] = useState(0);
  const [isDeleting, setIsDeleting] =
```

Código de TaskDetailPage.tsx fragmento 2 de 5

```
    useState(false);

  useEffect(() => {
    if (!taskId) {
      setStatus('not-found');
      return;
    }

    const controller = new AbortController();

    async function loadTask(): Promise<void> {
      setStatus('loading');
      setErrorMessage('');

      try {
        const response = await getTaskById(
          taskId as string,
          controller.signal,
        );
        setTask(response.data);
        setStatus('success');
      } catch (error) {
        if (isAbortError(error)) {
          return;
        }

        if (isHttpError(error) && error.status === 404) {
          setStatus('not-found');
          return;
        }

        setStatus('error');
        setErrorMessage(getHttpErrorMessage(error));
      }
    }

    void loadTask();

    return () => controller.abort();
```

Código de TaskDetailPage.tsx fragmento 3 de 5

```
  }, [taskId, reloadKey]);

  async function handleDelete(): Promise<void> {
    if (!task) {
      return;
    }

    const shouldDelete = window.confirm(
      `¿Eliminar la tarea "${task.title}"?`,
    );

    if (!shouldDelete) {
      return;
    }

    setIsDeleting(true);

    try {
      await deleteTask(task.id);
      navigate('/tasks', { replace: true });
    } catch (error) {
      setStatus('error');
      setErrorMessage(getHttpErrorMessage(error));
      setIsDeleting(false);
    }
  }

  if (status === 'loading') {
    return <p role="status">Consultando tarea...</p>;
  }

  if (status === 'not-found') {
    return (
      <NotFoundPage
        title="Tarea no encontrada"
        message="La tarea no existe o fue eliminada."
      />
    );
  }
```

Código de TaskDetailPage.tsx fragmento 4 de 5

```
if (status === 'error' || !task) {
  return (
    <section className="panel" role="alert">
      <h2>No fue posible consultar la tarea</h2>
      <p>{errorMessage}</p>
      <button
        type="button"
        onClick={() =>
          setReloadKey((current) => current + 1)
        }
      >
        Reintentar
      </button>
    </section>
  );
}

return (
  <article
    className="panel task-detail"
    aria-labelledby="task-detail-title"
  >
    <p className="section-heading__eyebrow">
      Detalle remoto
    </p>
    <h2 id="task-detail-title">{task.title}</h2>

    <dl className="task-detail__grid">
      <div>
        <dt>Identificador</dt>
        <dd>{task.id}</dd>
      </div>
      <div>
        <dt>Estado</dt>
        <dd>
          {task.status === 'completed'
            ? 'Completada'
            : 'Pendiente'}
```

Código de TaskDetailPage.tsx fragmento 5 de 5

```
          </dd>
        </div>
        <div>
          <dt>Fecha de creación</dt>
          <dd>
            <time dateTime={task.createdAt}>
              {formatTaskDate(task.createdAt)}
            </time>
          </dd>
        </div>
        <div>
          <dt>Última actualización</dt>
          <dd>
            <time dateTime={task.updatedAt}>
              {formatTaskDate(task.updatedAt)}
            </time>
          </dd>
        </div>
      </dl>

      <div className="task-detail__actions">
        <Link
          className="button-link button-link--secondary"
          to="/tasks"
        >
          Volver a tareas
        </Link>
        <button
          type="button"
          className="button-danger"
          disabled={isDeleting}
          onClick={() => void handleDelete()}
        >
          {isDeleting ? 'Eliminando...' : 'Eliminar tarea'}
        </button>
      </div>
    </article>
  );
}
```

El detalle ya no depende de que la tarea esté incluida en la página actual. Por eso una recarga directa de /tasks/:taskId continúa funcionando.

Paso 26 Simplificar App

Propósito. Declarar las rutas sin administrar tareas ni localStorage.

Archivo. src/App.tsx

Acción. Reemplaza todo el contenido.

Código de App.tsx

```
import { Route, Routes } from 'react-router';
import { AppLayout } from './layouts/AppLayout';
import { HomePage } from './pages/HomePage';
import { NotFoundPage } from './pages/NotFoundPage';
import { TaskDetailPage } from './pages/TaskDetailPage';
import { TasksPage } from './pages/TasksPage';

export default function App() {
  return (
    <Routes>
      <Route element={<AppLayout />}>
        <Route index element={<HomePage />} />
        <Route path="tasks" element={<TasksPage />} />
        <Route
          path="tasks/:taskId"
          element={<TaskDetailPage />}
        />
        <Route path="*" element={<NotFoundPage />} />
      </Route>
    </Routes>
  );
}
```

App vuelve a concentrarse en navegación. TasksPage administra la consulta remota y TaskDetailPage administra la recuperación de un recurso específico.

Parte E Estilos y limpieza

Paso 27 Agregar estilos para las solicitudes

Propósito. Mantener visibles los estados y adaptar paginación y mensajes.

Archivo. src/styles/index.css

Acción. Conserva los estilos anteriores y agrega el bloque al final, antes de la regla responsiva si existe.

Código que se agrega a index.css

```
.request-status,
.operation-message {
  margin: 1rem 0;
  padding: 0.9rem 1rem;
  border: 1px solid #cbd5e1;
  border-radius: 0.5rem;
  background: #f8fafc;
}

.request-status--error,
.operation-message--error {
  border-color: #b91c1c;
}

.request-status h3 {
  margin-top: 0;
}

.task-summary {
  display: flex;
  flex-wrap: wrap;
  justify-content: space-between;
  gap: 0.75rem;
}

.pagination {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 1rem;
  margin-top: 1.25rem;
}

[aria-busy='true'] {
  opacity: 0.7;
}

@media (max-width: 640px) {
  .task-summary,
  .pagination {
    align-items: stretch;
    flex-direction: column;
  }
}
```

Paso 28 Retirar la fuente local

Propósito. Evitar dos fuentes de verdad y código que ya no participa en la aplicación.

Ubicación. Raíz del proyecto Frontend.

Acción. Después de comprobar que no existen importaciones, elimina únicamente los módulos locales sustituidos.

Comprobación y eliminación

```
git grep -n -E "data/tasks|task-storage|task-query" -- src

git rm src/data/tasks.ts
git rm src/services/task-storage.service.ts
git rm src/services/task-query.service.ts
```

Si git grep muestra una importación activa, corrígela antes de ejecutar git rm. No elimines task.interface.ts, task-validation.ts ni los componentes compartidos.

Paso 29 Comprobar la estructura final

Propósito. Verificar que configuración, contratos, servicios, hook, páginas y componentes estén separados.

Ubicación. Explorador de Visual Studio Code.

Acción. Compara el proyecto con la estructura esperada.

Estructura esperada

```
task-manager-frontend/
|-- docs/
|-- public/
|-- src/
|   |-- components/
|   |   |-- AppNavigation.tsx
|   |   |-- AsyncStatus.tsx
|   |   |-- TaskFilters.tsx
|   |   |-- TaskForm.tsx
|   |   |-- TaskItem.tsx
|   |   |-- TaskList.tsx
|   |   |-- TaskPagination.tsx
|   |   `-- TaskSummary.tsx
|   |-- config/
|   |   `-- env.ts
|   |-- hooks/
|   |   `-- useTaskList.ts
|   |-- layouts/
|   |   `-- AppLayout.tsx
|   |-- models/
|   |   |-- api.types.ts
|   |   |-- request-status.type.ts
|   |   |-- task-view.types.ts
|   |   `-- task.interface.ts
|   |-- pages/
|   |   |-- HomePage.tsx
|   |   |-- NotFoundPage.tsx
|   |   |-- TaskDetailPage.tsx
|   |   `-- TasksPage.tsx
|   |-- services/
|   |   |-- http.service.ts
|   |   `-- task.api.service.ts
|   |-- styles/
|   |   `-- index.css
|   |-- utils/
|   |   |-- date.ts
|   |   `-- task-validation.ts
|   |-- App.tsx
|   |-- main.tsx
|   `-- vite-env.d.ts
|-- .env.example
|-- .gitignore
|-- package.json
|-- README.md
`-- vite.config.ts
```

Parte F Comprobación guiada

Paso 30 Iniciar los dos proyectos

Propósito. Mantener disponible la API y ejecutar el Frontend mediante el proxy.

Ubicación. Dos terminales independientes.

Acción. Inicia primero el Backend y después el Frontend.

Terminal del Backend

```
cd task-api-express
pnpm dev
```

Terminal del Frontend

```
cd task-manager-frontend
pnpm dev
```

No cierres la terminal del Backend. Abre la aplicación mediante la dirección que muestre Vite, normalmente http://localhost:5173.

Paso 31 Observar la primera consulta

Propósito. Relacionar la interfaz con la pestaña Network del navegador.

Ubicación. Ruta /tasks y herramientas de desarrollo.

Acción. Recarga la página y localiza la solicitud GET.

1. Abre Network y filtra por Fetch/XHR.
2. Recarga /tasks.
3. Confirma GET /api/tasks con page, limit, sortBy y order.
4. Comprueba el código 200.
5. Revisa que Response contenga data y meta.
6. Confirma que la pantalla presente las mismas tareas que la respuesta.

Paso 32 Crear una tarea remota

Propósito. Comprobar POST y persistencia en Atlas.

Ubicación. Formulario de /tasks, Network y Atlas Data Explorer.

Acción. Registra una tarea de demostración.

1. Escribe Preparar demostración de Fetch.
2. Presiona Agregar una sola vez.
3. Comprueba POST /api/tasks con código 201.
4. Confirma que el formulario se limpie después de la respuesta.
5. Recarga el navegador y verifica que la tarea permanezca.
6. Comprueba el documento correspondiente en Atlas.

Paso 33 Actualizar título y estado

Propósito. Comprobar PATCH y sincronización posterior.

Ubicación. Tarjeta de la tarea y Network.

Acción. Edita el título, completa la tarea y vuelve a abrirla.

1. Selecciona Editar, cambia el título y guarda.
2. Comprueba PATCH /api/tasks/:id con title.
3. Selecciona Completar y comprueba PATCH con status completed.
4. Selecciona Reabrir y comprueba status pending.
5. Recarga la aplicación y confirma que el último estado persista.

Paso 34 Eliminar una tarea

Propósito. Comprobar confirmación, DELETE 204 y actualización de la página.

Ubicación. Lista y Network.

Acción. Cancela una eliminación y después confirma otra.

1. Presiona Eliminar y selecciona Cancelar. No debe existir solicitud DELETE.
2. Repite y confirma la operación.
3. Comprueba DELETE /api/tasks/:id con código 204 y cuerpo vacío.
4. Confirma que la tarea desaparezca después de actualizar la consulta.
5. Si era el último registro de una página posterior, comprueba que la interfaz regrese a la página anterior.

Paso 35 Probar consulta y paginación

Propósito. Verificar que la API, no React, ejecute búsqueda, filtro, orden y división en páginas.

Ubicación. Controles de /tasks y Network.

Acción. Ejecuta cada combinación y observa la URL solicitada.

Acción	Parámetros esperados
Buscar api	search=api
Mostrar completadas	status=completed
Título A-Z	sortBy=title&order=asc
Página siguiente	page=2&limit=5
Combinar criterios	search, status, sortBy, order, page y limit

Al cambiar búsqueda, filtro u orden, page debe volver a 1. La respuesta meta controla los botones Anterior y Siguiente.

Paso 36 Probar la ruta de detalle

Propósito. Confirmar que el recurso se consulta por id y no depende de la lista cargada.

Ubicación. Ruta /tasks/:taskId.

Acción. Abre un detalle, recarga y prueba identificadores inexistentes.

1. Selecciona Ver detalle y comprueba GET /api/tasks/:id.
2. Recarga directamente la ruta dinámica y confirma que la tarea vuelva a mostrarse.
3. Sustituye el id por uno válido en formato, pero inexistente, y comprueba la respuesta 404.
4. Elimina una tarea desde el detalle y confirma la navegación hacia /tasks.

Paso 37 Probar una falla y el reintento

Propósito. Comprobar que la interfaz no confunda una falla de red con una colección vacía.

Ubicación. Terminal del Backend y ruta /tasks.

Acción. Detén temporalmente la API, ejecuta una consulta y recupérala.

1. Detén el Backend con Ctrl+C.
2. Cambia un filtro o recarga /tasks.
3. Confirma que aparezca un mensaje de error comprensible.
4. Comprueba que aparezca Reintentar.
5. Inicia nuevamente el Backend.
6. Presiona Reintentar y verifica que la colección se recupere.

Paso 38 Comprobar la cancelación

Propósito. Observar que las consultas rápidas no dejan resultados fuera de orden.

Ubicación. Network y campo Buscar.

Acción. Escribe y borra varios caracteres rápidamente.

Algunas solicitudes pueden aparecer como canceladas. Esto es correcto: AbortController detiene las consultas obsoletas y la pantalla conserva el resultado correspondiente al criterio más reciente.

Paso 39 Ejecutar validaciones técnicas

Propósito. Detectar tipos, importaciones y errores de compilación.

Ubicación. Raíz del Frontend.

Acción. Detén el servidor si es necesario y ejecuta los comandos disponibles en package.json.

Comandos mínimos

```
pnpm build
pnpm lint
```

Si package.json no contiene el script lint, ejecuta pnpm build y no inventes el comando. La consola del navegador y la terminal deben permanecer sin errores durante las pruebas normales.

8 Pruebas funcionales

Prueba	Acción	Resultado esperado
Listado	Abrir /tasks	GET 200 y tareas visibles
Carga	Reducir temporalmente la velocidad	Mensaje de consulta en curso
Vacío	Buscar texto inexistente	data vacío y mensaje sin resultados
Creación	Registrar título válido	POST 201 y tarea persistida
Validación	Enviar título vacío	No se realiza POST
Edición	Cambiar el título	PATCH 200 y texto actualizado
Estado	Completar y reabrir	PATCH 200 en ambas acciones
Eliminación cancelada	Cancelar confirmación	No existe DELETE
Eliminación confirmada	Aceptar confirmación	DELETE 204 y lista actualizada
Detalle	Abrir y recargar /tasks/:id	GET por id y datos visibles
No encontrada	Consultar id inexistente	Vista Tarea no encontrada
Paginación	Cambiar de página	meta y controles coherentes
Red	Detener API	Error comprensible y Reintentar
Recuperación	Reiniciar API y reintentar	La lista vuelve a mostrarse
Persistencia	Recargar navegador	Los cambios permanecen en Atlas
Compilación	Ejecutar pnpm build	Termina sin errores

9 Errores frecuentes

Problema	Causa probable	Revisión
Falta VITE_API_URL	.env.local no existe o Vite no reinició	Crear el archivo y reiniciar pnpm dev
404 en /api/tasks	Proxy o ruta base incorrectos	Revisar vite.config.ts y VITE_API_URL
Failed to fetch	Backend detenido o proxy no activo	Probar /health y reiniciar ambos proyectos
CORS en el navegador	Se usó http://localhost:3000 directamente	Restaurar VITE_API_URL=/api durante desarrollo
response.json falla	Se intentó leer DELETE 204	Conservar el caso response.status === 204
Filtros devuelven 422	Valor no permitido o parámetro vacío	Revisar TaskListQuery y URLSearchParams
La lista no se actualiza	No se llamó reload después de una mutación	Revisar el controlador de la operación
Resultados cambian hacia atrás	Una solicitud anterior terminó después	Conservar AbortController y cleanup
El detalle falla al recargar	Busca solo dentro de la página actual	Usar getTaskById en TaskDetailPage
El formulario se limpia al fallar	No espera el resultado booleano	Limpiar solo cuando wasCreated sea true
Se muestran _id o __v	Backend no transforma documentos	Corregir toTask en el servicio Backend
La aplicación usa localStorage	Persisten imports o efectos anteriores	Retirar el servicio local y sus importaciones

Parte G Documentación y control de versiones

Paso 40 Actualizar README

Propósito. Documentar la ejecución coordinada y el contrato remoto.

Archivo. README.md

Acción. Conserva la documentación anterior y agrega una sección equivalente.

Contenido recomendado para README.md

````
## Integración con la API REST

La aplicación consume la API de tareas mediante Fetch API.

### Requisitos

- Backend disponible en http://localhost:3000
- Frontend ejecutado mediante Vite
- MongoDB Atlas conectado desde el Backend

### Configuración local

1. Copiar `.env.example` como `.env.local`.
2. Confirmar `VITE_API_URL=/api`.
3. Iniciar el Backend con `pnpm dev`.
4. Iniciar el Frontend con `pnpm dev`.

### Funciones remotas

- Listado paginado.
- Búsqueda, filtro y ordenamiento.
- Creación, edición y cambio de estado.
- Eliminación con confirmación.
- Consulta de detalle por identificador.
- Estados de carga, vacío, error y reintento.

### Compilación

```bash
pnpm build
```
````

Paso 41 Registrar evidencias

Propósito. Conservar resultados reales sin publicar credenciales ni direcciones de Atlas.

Archivo. docs/pruebas-api-rest-ec3-a10.md

Acción. Crea el archivo y completa cada espacio después de ejecutar las pruebas.

Plantilla de evidencias

```
# Pruebas de API REST - EC3 F2 A10

## Entorno

- Rama:
- Dirección pública usada por el Frontend: /api
- Resultado de pnpm build:

## Casos ejecutados

| Caso | Método y ruta | Esperado | Obtenido | Resultado |
|---|---|---:|---:|---|
| Listar | GET /api/tasks | 200 | | |
| Crear | POST /api/tasks | 201 | | |
| Actualizar | PATCH /api/tasks/:id | 200 | | |
| Eliminar | DELETE /api/tasks/:id | 204 | | |
| Inexistente | GET /api/tasks/:id | 404 | | |
| API detenida | GET /api/tasks | Error de red | | |

## Evidencias

- Captura de la lista:
- Captura de Network:
- Captura del error y reintento:
- Observaciones:

No incluir URI, usuario ni contraseña de MongoDB Atlas.
```

Paso 42 Responder las preguntas de cierre

Propósito. Explicar las decisiones técnicas con palabras propias.

Archivo. docs/preguntas-cierre-ec3-a10.md

Acción. Crea el archivo, copia las preguntas y responde debajo de cada una.

1. ¿Qué diferencia existe entre una falla de red y una respuesta HTTP 404?
2. ¿Por qué debe comprobarse response.ok después de fetch?
3. ¿Qué responsabilidad tiene http.service.ts?
4. ¿Qué responsabilidad tiene task.api.service.ts?
5. ¿Por qué VITE_API_URL no debe contener credenciales?
6. ¿Qué problema resuelve el proxy de Vite durante el desarrollo?
7. ¿Por qué el proxy no sustituye la configuración de producción?
8. ¿Qué función cumple AbortController?
9. ¿Por qué la ruta de detalle consulta la tarea por id?
10. ¿Por qué el formulario se limpia únicamente después de un POST satisfactorio?
11. ¿Qué ventaja ofrece separar la consulta remota de los componentes visuales?
12. ¿Qué cambio fue más difícil y cómo comprobaste que funcionaba?

Estructura del archivo

```
# Preguntas de cierre - EC3 F2 A10

## 1. ¿Qué diferencia existe entre una falla de red y una respuesta HTTP 404?

Escribe aquí tu respuesta.

## 2. ¿Por qué debe comprobarse response.ok después de fetch?

Escribe aquí tu respuesta.
```

Paso 43 Registrar la implementación

Propósito. Conservar un historial descriptivo y publicable.

Ubicación. Raíz del Frontend.

Acción. Revisa archivos, confirma que .env.local esté ignorado y registra los cambios.

Comandos

```
git status
git check-ignore .env.local
git add .
git commit -m "feat: integrar api rest y crud remoto"
git push -u origin feature/api-rest-crud-remoto
```

Antes del commit ejecuta git status y confirma que .env.local no aparezca preparado. Nunca publiques credenciales, URI de Atlas ni capturas que las muestren.

10 Evidencias requeridas

1. Rama feature/api-rest-crud-remoto.
2. VITE_API_URL declarada sin secretos.
3. Proxy local configurado.
4. Tipos de respuesta y metadatos.
5. Cliente HTTP con response.ok y 204.
6. Servicio de tareas con GET, POST, PATCH y DELETE.
7. Hook de consulta con AbortController.
8. Estados loading, success, empty y error.
9. Botón de reintento funcional.
10. Creación, edición, estado y eliminación persistentes.
11. Búsqueda, filtro, orden y paginación remotos.
12. Detalle consultado por id.
13. Código local obsoleto retirado.
14. README y evidencias actualizados.
15. Preguntas de cierre respondidas.
16. pnpm build sin errores.
17. Repositorio remoto actualizado.

11 Criterios de evaluación

Criterio	Evidencia esperada
Configuración	Variable pública y proxy funcionan sin exponer secretos
Contrato	Tipos coinciden con data, meta y errores del Backend
Cliente HTTP	Centraliza Fetch, JSON, response.ok, 204 y errores
Servicio de tareas	Métodos y rutas no se repiten en componentes
Consulta	GET usa parámetros validados y metadatos remotos
CRUD	POST, PATCH y DELETE persisten y refrescan la interfaz
Asincronía	Promesas se esperan y las solicitudes obsoletas se cancelan
Estados visuales	Carga, vacío, error y reintento son comprensibles
Detalle	La ruta dinámica funciona después de recargar
Navegación	Inicio, Tareas, Detalle y 404 continúan funcionando
Accesibilidad	Etiquetas, roles, estados y controles son utilizables
Calidad	TypeScript sin any innecesario y consola sin errores
Documentación	README, pruebas y respuestas están completos
Control de versiones	Commit descriptivo y archivo local protegido
Compilación	pnpm build termina correctamente

12 Lista de verificación

☐ Recuperé y compilé la versión final de la Actividad 9.
☐ Comprobé la API y MongoDB Atlas antes de modificar React.
☐ Creé la rama de integración.
☐ Configuré .env.example y .env.local.
☐ Confirmé que .env.local está ignorado.
☐ Configuré el proxy de Vite.
☐ Definí los contratos HTTP.
☐ Creé http.service.ts y task.api.service.ts.
☐ Implementé useTaskList con cancelación.
☐ Mostré carga, vacío, error y reintento.
☐ Implementé CRUD remoto.
☐ Conecté búsqueda, filtro, orden y paginación.
☐ Actualicé el detalle para consultar por id.
☐ Retiré localStorage como fuente de tareas.
☐ Ejecuté las pruebas positivas y negativas.
☐ Revisé Network y la consola.
☐ Ejecuté pnpm build.
☐ Actualicé README y evidencias.
☐ Respondí las preguntas de cierre.
☐ Registré y publiqué los cambios.

Parte H Desafío individual

Conservar la última consulta en la URL

Representa búsqueda, estado, orden y página mediante useSearchParams para que una consulta pueda compartirse o recuperarse después de recargar. Este desafío no forma parte de las evidencias obligatorias.

- Utilizar useSearchParams de React Router.
- Validar los valores antes de construir TaskListQuery.
- Omitir parámetros que utilicen el valor predeterminado.
- Conservar el funcionamiento de Atrás y Adelante.
- No agregar otra dependencia.

13 Cierre de la actividad

Al finalizar, el Administrador de tareas deja de utilizar el navegador como fuente de verdad y opera sobre la API REST del proyecto Backend. La interfaz conserva navegación, validación y componentes, pero ahora representa correctamente la latencia, los errores y la persistencia remota. Esta base permitirá agregar autenticación, sesión y rutas protegidas en la Actividad 11.

14 Fuentes de consulta

- MDN Web Docs. Uso de Fetch API. https://developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch
- MDN Web Docs. Response.ok. https://developer.mozilla.org/es/docs/Web/API/Response/ok
- MDN Web Docs. AbortController. https://developer.mozilla.org/es/docs/Web/API/AbortController
- React. useEffect. https://react.dev/reference/react/useEffect
- React. Sincronización con efectos. https://react.dev/learn/synchronizing-with-effects
- Vite. Variables de entorno y modos. https://vite.dev/guide/env-and-mode
- Vite. Opciones del servidor y proxy. https://vite.dev/config/server-options
- React Router 7.18.4. useParams. https://reactrouter.com/api/hooks/useParams
- Guía Backend EC2 F3 A8. Arquitectura, paginación, filtros, búsqueda y ordenamiento.